Hot Keys

one key, one node, and a million readers

Watch the 4-min video

In one line: sharding spreads keys, not traffic: one very popular key lives on one node and can saturate it; detect hot keys with metrics or sampling, then serve them from a short-lived local (L1) cache on each app server, copy them under several suffixes across shards, or read them from replicas, accepting a few seconds of staleness.

1 A celebrity posts

A celebrity posts, and their profile is read a million times a minute. All those reads hit one cache key, which lives on one Redis node. That node's CPU hits 100% while the rest of the cluster sits idle. Adding more nodes doesn't help, because the key can only live in one place. That's the **hot key** problem.

2 Why sharding doesn't save you

Sharding (consistent hashing in lesson 5, Redis Cluster slots in lesson 28) spreads *keys* across nodes, but each key maps to exactly **one** node. So the cluster's capacity for a single key is the capacity of one node, maybe 100,000 simple operations per second, less with big values, which also saturate the node's network. When one key gets more traffic than that, its node becomes the bottleneck, and every other key on it suffers too.

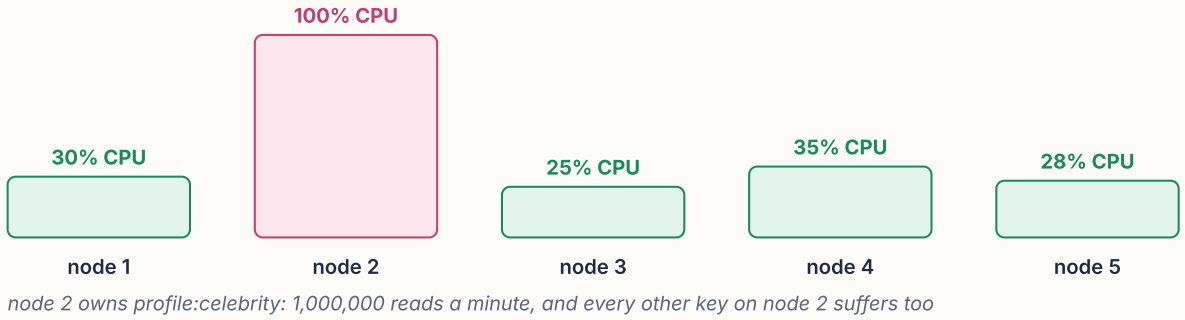


Figure 1. One hot key: its node is saturated while the others idle.

3 Detect them first

- `redis-cli --hotkeys` (needs an LFU eviction policy, so Redis tracks access frequency).
- Sampling with `MONITOR`, carefully: it's expensive on a busy server, so only for seconds.
- Counting key accesses in the client library or a proxy, and exporting the top-N as metrics.
- Alerting when one node's CPU or network is far above its peers.

4 Fix 1: a local in-process cache (L1)

The most effective fix: keep the hottest keys in memory **on each application server** for a short time, 1 to 5 seconds. With 200 app servers and a 2-second L1 TTL, the Redis node sees at most about 200 requests every 2 seconds for that key, instead of a million a minute. The trade-off is a few seconds of staleness. Lesson 32 covers multi-level caching in general.

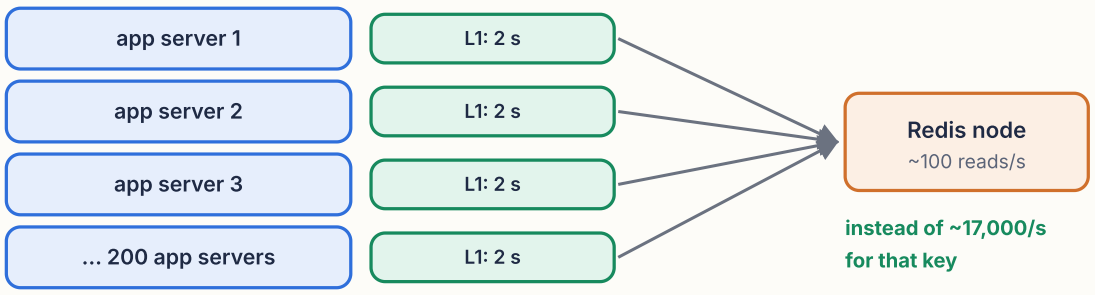


Figure 2. A 2-second in-process cache on every app server shrinks the hot key's load on Redis by about 170 times.

5 Fix 2: replicate the key under suffixes

Store copies of the hot value under several keys, like `profile:42#1` to `profile:42#10`. Each lands on a different shard. Readers pick a random suffix, spreading the load across ten nodes. Writes must update or delete **all** copies, so this suits read-mostly keys.

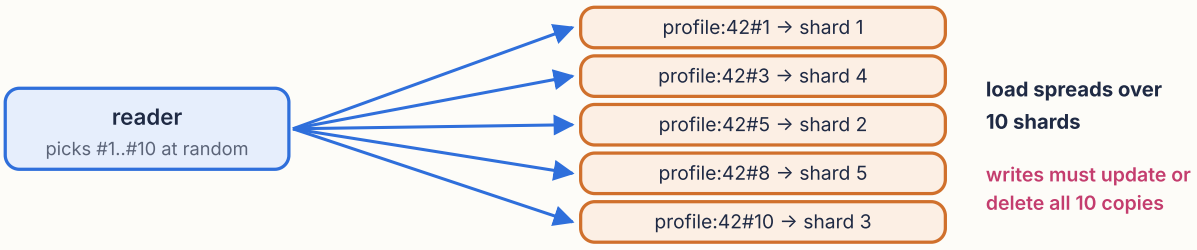


Figure 3. Suffix replication: ten copies on ten shards, chosen at random by readers.

6 Fix 3: read replicas and smart proxies

Redis replicas can serve reads (`READONLY` in cluster mode), so a hot key's reads spread across the primary and its replicas, with a little replication lag. Some managed caches and proxies detect hot keys automatically and cache them closer to the client.

WORKED EXAMPLE: THE CELEBRITY PROFILE

A million reads per minute is about **17,000 per second**. Each of 200 app servers caches the profile locally for 2 seconds, so Redis sees about **100 reads per second** for that key. The key is also replicated under ten suffixes as a backup if L1 is bypassed. When the celebrity updates their profile, all copies are deleted, and readers accept up to 2 seconds of staleness.

COMMON MISTAKES

- Adding cache nodes to fix a hot key: it still lands on one of them.
- Huge values on a hot key (a 1 MB JSON read 10,000 times a second is 10 GB/s of network).
- Forgetting that hot *writes* (a global counter) need a different fix: shard the counter and sum the parts.

7 Key terms

Hot key: one key with far more traffic than others

L1 / local cache: short-lived in-process cache on each app server

Hot shard: the node that holds a hot key

Suffix replication: copies under key#1..key#n on different shards

--hotkeys: redis-cli option to list hot keys (LFU needed)

Sharded counter: split a hot counter into parts, sum on read

8 Interview questions

Q1. How do you handle a single extremely hot cache key? BASIC

Detect it with metrics or hotkeys sampling, add a short-lived local in-process cache on app servers, replicate the key under several suffixes across shards, and use read replicas, accepting a few seconds of staleness.

Q2. Why doesn't adding nodes help? BASIC

Because one key always maps to one shard, so its load can't spread unless you copy it.

Q3. A flash sale item gets 200,000 reads per second. Design the read path. SCENARIO

Serve it from an in-process cache refreshed every second or two on every app server (and a CDN for the page), with Redis behind it; replicate the Redis key under suffixes; never let the key expire under load.

Q4. A global 'likes' counter key is hot for writes. What do you do? SCENARIO

Shard the counter: increment one of N sub-keys chosen at random, and sum them on read (cached briefly), or aggregate in memory and flush periodically (write-behind).

Q5. What's the cost of an L1 cache on every app server? TRICKY

Staleness up to its TTL, memory on every server, and inconsistency between servers during that window; for most hot read keys a 1 to 5 second bound is acceptable.

Q6. How would you find hot keys in production without slowing Redis? TRICKY

Enable an LFU policy and use redis-cli --hotkeys, or count accesses in the client or proxy and export top-N metrics; avoid long MONITOR sessions on busy servers.

Next: SD-F-013 · Redis Fundamentals backendroadmap.com